

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«АЛТАЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных технологий
Кафедра информатики

**Разработка backend-части информационной системы управления
мероприятиями**
выпускная квалификационная работа

Выполнил:
студент группы 4.205-2,
Шацких Алексей Евгеньевич

(подпись)

Научный руководитель:
к.т.н., доцент
Михеева Татьяна Викторовна

(подпись)

Работа защищена:
«__» _____ 2026 г.

Оценка: _____

Председатель ГЭК:
д.т.н., профессор
Леонов Сергей Леонидович

(подпись)

Допустить к защите:
Зав. кафедрой, к.ф.-м.н., доцент
Козлов Денис Юрьевич

(подпись)

«__» _____ 2026 г.

Барнаул 2026

РЕФЕРАТ

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ BACKEND РАЗРАБОТКИ КЛИЕНТ- СЕРВЕРНЫХ ПРИЛОЖЕНИЙ	6
1.1. Методологии и подходы backend-разработки	6
1.2. Архитектурные подходы и паттерны проектирования backend	12
1.3. Анализ и выбор программных средств разработки	14
2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ	15
2.1. Расчёт экономической выгоды	15
2.2. Проектирование базы данных	15
3. РАЗРАБОТКА BACKEND-ЧАСТИ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ	16
3.1. Разработка доменного слоя	16
3.2. Разработка инфраструктурного слоя	16
3.3. Разработка слоя бизнес-логики	16
3.4. Разработка слоя API	16
ЗАКЛЮЧЕНИЕ	17
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	18
ПРИЛОЖЕНИЕ	20

ВВЕДЕНИЕ

Мероприятия различного вида проводятся каждый день в миллионах организаций по всему миру. Для проведения мероприятия организующему необходимо выбрать место, создать анонсы в мессенджерах и социальных сетях, узнать, какое количество человек планирует посетить его, возможно, создать таблицу, чтобы участники записывались в неё. Из-за того, что таких мероприятий в крупных организациях проводится большое количество, у организаторов возникают трудности в информационной поддержке организационных процессов, чтобы мероприятие состоялось.

Для решения этих проблем мы предлагаем создать информационную систему мероприятий, которая была бы встраиваемой в уже существующую экосистему организации, и позволяющую контролировать все процессы организации мероприятий в одном месте.

Актуальность темы обусловлена растущей цифровизацией всех отраслей экономики, что влечёт за собой увеличение потребности рынка в новых цифровых решениях привычных и рутинных задач.

Разработка предлагаемой информационной системы требует применения современных подходов и технологий, таких как использование паттернов проектирования, фреймворков, технологий тестирования и баз данных.

Данная выпускная квалификационная работа посвящена разработке backend-части системы, которая является ключевой составляющей всей системы, обеспечивая её стабильное функционирование, обработку бизнес-логики, хранение данных и интеграцию с внешними сервисами.

Целью данной выпускной квалификационной работы является разработка backend-части информационной системы.

Для достижения поставленной цели необходимо решить следующие **задачи:**

1. Изучить основные методы и технологии backend-разработки.
2. Выбрать архитектурные принципы и паттерны проектирования.
3. Выбрать программные средства разработки.
4. Спроектировать базу данных.
5. Разработать доменный слой.
6. Разработать инфраструктурный слой.
7. Разработать слой бизнес-логики.
8. Разработать слой API.

Объектом исследования данной работы является методология и практика разработки клиент-серверных приложений.

Предмет исследования – backend разработка с применением фреймворка ASP.NET.

Практическая значимость разработки backend-части системы управления мероприятиями заключается в создании поддерживаемой, расширяемой, масштабируемой и отказоустойчивой инфраструктуры системы.

Разрабатываемая система позволит упростить информационную поддержку процессов проведения мероприятий, что позволит увеличить вовлечённость участников.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ BACKEND РАЗРАБОТКИ КЛИЕНТ-СЕРВЕРНЫХ ПРИЛОЖЕНИЙ

1.1. Методологии и подходы backend-разработки

При разработке backend-приложений, как и при разработке любого программного обеспечения, применяются различные методологии и подходы, позволяющие упростить разработку, поддержку, уменьшить количество ошибок в программном коде, а также увеличить взаимопонимание между заказчиком и командой разработки. Рассмотрим некоторые из них.

Test-Driven Development (TDD) [1] – методология, которая предполагает разработку программного обеспечения через тестирование. Процесс разработки по данной методологии происходит через небольшие итерации, проходящие в три этапа.

Первым этапом, именуемым «красной зоной», является написание тестов для ещё не существующего функционала, которые будут выдавать ошибку до того момента, пока не будет написана реализация.

Вторым этапом, именуемым «зелёной зоной», является написание реализации функционала, который будет проходить тесты без ошибок. Данный этап самый короткий и, если функционал проходит тесты, то он считается реализованным.

Третьим этапом, именуемым «синей зоной», является рефакторинг, то есть изменение и улучшение уже существующего функционала и тестов. В данной методологии рефакторинг является безопасным процессом, поскольку ошибки, возникающие при тестировании, содержат в себе точную информацию о месте и причине возникновения.

К достоинствам методологии относится создание тестируемого и легко поддерживаемого кода: поскольку тесты пишутся до реализации, разработчик вынужден проектировать модули с минимальной связностью, что улучшает архитектуру системы. Непрерывно пополняемый набор регрессионных тестов позволяет обнаруживать нарушения существующей функциональности сразу после внесения изменений. Помимо этого, тесты выполняют роль живой документации, точно описывая ожидаемое поведение каждого компонента.

К недостаткам относится замедление разработки на начальном этапе, пока команда не выработала устойчивый навык работы по данной методологии. Дополнительную сложность представляет тестирование кода, взаимодействующего с внешними системами, что требует применения макетов (mock-объектов). При нестабильных требованиях тесты нередко нуждаются в переработке наравне с реализацией, что может нивелировать часть выигрыша от раннего обнаружения ошибок.

Domain-Driven Design (DDD) [2] – это подход проектирования объектов программного обеспечения как сущностей реального мира, с их бизнес-логикой и жизненным циклом, отсюда проистекает определение данного подхода как «предметно-ориентированное программирование». Рассмотрим основные правила, которые предлагает данный подход.

Первым правилом, которое предлагает DDD, является разделение объектов предметной области на сущности и объекты-значения.

Объекты-значения (value-objects) – это маленькие составные блоки предметной области, которые не являются уникальными, не имеют своей бизнес-логики, жизненного цикла и могут сравниваться по значению. Объекты-значения предполагаются неизменяемыми, поэтому вместо изменения уже

существующего объекта класса предлагается создавать новый объект. Рекомендуется, чтобы объекты-значения ссылались только на другие объекты-значения посредством объектов классов.

Сущности (entities) – это объекты предметной области, которые составлены из объектов-значений, они имеют свою бизнес-логику, жизненный цикл и являются уникальными, поэтому для их однозначного определения вводится идентификатор, который становится операндом сравнения. Сущности предполагаются изменяемыми, но рекомендуется инкапсулировать логику их изменения в публичных методах, не допуская прямого изменения полей объекта класса. Также рекомендуется, чтобы сущности ссылались на объекты-значения и другие сущности посредством объектов классов.

Вторым правилом, которое предлагает DDD, является выделение связанных между собой сущностей и объектов-значений в отдельные кластеры, называемые агрегатами. В каждом агрегате предлагается выделить главенствующую сущность, называемую корнем агрегата (aggregate root), которая будет отвечать за управление связанными сущностями и объектами-значениями. Рекомендуется, чтобы корень агрегата ссылался на другие корни агрегатов через идентификаторы.

Третьим правилом, которое предлагает DDD, является привнесение в доменную область репозитория (repositories) – объектов, которые отвечают за сохранение сущности в программном обеспечении. Как правило, в доменной области определяется только интерфейс репозитория, а непосредственно реализация этого интерфейса происходит в инфраструктурном слое приложения.

Четвёртым правилом, которое предлагает DDD, является выделение логики создания связанных сущностей в отдельные фабрики, что позволяет решить проблему разграничения зон ответственности сущностей.

К достоинствам данного подхода относится инкапсуляция бизнес-логики в рамках единой доменной модели, что упрощает её повторное использование и перенос в между проектами, упрощение проектирования для разработчиков, не являющихся экспертами в предметной области, увеличение понимания между заказчиком и командой разработки, посредством вырабатывания единого языка (Ubiquitous Language).

К недостаткам данного подхода относится необходимость высокой квалификации разработчиков для её эффективного применения, неприменимость без соответствующих архитектурных решений, что влечёт за собой увеличение затрат на первоначальное проектирование и увеличение итоговой стоимости проекта. По этим причинам DDD подходит проектам со сложной и долгоживущей бизнес-логикой, тогда как для небольших и краткосрочных проектов затраты на применение подхода, как правило, не окупаются.

Behaviour-Driven Development (BDD) [3] – подход к разработке программного обеспечения, являющийся эволюцией метода Test-Driven Development. Основной принцип подхода состоит в том, что до написания какой-либо реализации специалисты предметной области совместно с разработчиками и тестировщиками описывают желаемое поведение системы на структурированном естественном языке. Наиболее распространённым форматом такого описания является конструкция Given-When-Then: предусловие (Given) задаёт начальное состояние системы, событие (When)

описывает действие пользователя, а ожидаемый результат (Then) фиксирует наблюдаемое поведение, которое должна продемонстрировать система. Полученные сценарии преобразуются в автоматические тесты с помощью специализированных инструментов, таких как Cucumber [4], SpecFlow [5] или Behave [6], и служат одновременно исполняемой спецификацией и приёмочными критериями.

Подход решает ключевое ограничение TDD, при котором тесты остаются артефактом, понятным только разработчикам. В BDD сценарии становятся общим языком между заказчиком, аналитиком, тестировщиком и разработчиком, что снижает риск расхождения между реализованной функциональностью и бизнес-требованиями. Это достигается за счёт непосредственного участия представителей предметной области в формулировке сценариев на ранних стадиях разработки.

Вместе с тем применение BDD сопряжено с рядом издержек. Поддержание актуальности сценариев требует постоянных усилий при изменении требований. Внедрение подхода предполагает обучение всех участников процесса работе с предметно-ориентированным языком и инструментами автоматизации. Совокупность этих факторов делает BDD более ресурсоёмким по сравнению с традиционными подходами, однако данные затраты, как правило, окупаются в проектах с высокой сложностью бизнес-логики и требованиями к долгосрочной сопровождаемости.

Spec-Driven Development (SDD) [7] – новый подход, появившийся на фоне активного развития нейросетей и ИИ-агентов [8]. Суть подхода заключается в том, что перед написанием какой-либо реализации команда проекта определяет требования, поведение, технологии и ограничения

программного обеспечения, фиксируя всё это в файлах спецификации. Данные файлы становятся источником истины как для человека, так и для ИИ-агента, что позволяет устранить ключевые системные ограничения при работе с языковыми моделями.

Выделяют четыре основные проблемы, решаемые данным подходом.

Во-первых, ограниченность области и длительности задач: языковые модели демонстрируют деградацию качества при изменениях, затрагивающих более 3-5 файлов одновременно. SDD решает эту проблему посредством принудительной декомпозиции функциональности на атомарные задачи.

Во-вторых, отсутствие контекста о конкретной разрабатываемой функциональности: спецификации обеспечивают ИИ-агенту полное представление о требованиях, граничных случаях, критериях приёмки и точках интеграции ещё до начала генерации кода.

В-третьих, незнание стандартов и соглашений конкретного проекта: специальный файл конституции (`constitution.md`) кодифицирует технологический стек, правила именования, архитектурные решения, допустимые библиотеки и требования безопасности.

В-четвёртых, неконтролируемая автономия ИИ-агента: в рабочий процесс встраиваются обязательные контрольные точки проверки, на которых человек проверяет спецификацию, архитектурный план и декомпозицию задач перед запуском автоматической реализации.

Рабочий процесс SDD включает шесть последовательных этапов: формирование конституции проекта (Constitution), составление спецификации функциональности (Specify), устранение неоднозначностей (Clarify), архитектурное планирование (Plan), декомпозиция на задачи (Tasks)

и непосредственная реализация (Implement). Такая структура смещает проектирование и принятие ключевых решений на ранние стадии разработки, сокращая объём переработок и технический долг на поздних этапах.

Для разработки backend-части информационной системы управления мероприятиями был выбран DDD подход, поскольку он упростит моделирование бизнес-логики управления мероприятиями, сделает программный код поддерживаемым для дальнейшего развития, упростит модульное тестирование, а также позволит приобрести необходимые в современных реалиях навыки проектирования, построения архитектуры и использования паттернов.

1.2. Архитектурные подходы и паттерны проектирования backend

При разработке backend-приложений, как и при разработке любого программного обеспечения, применяются различные архитектурные подходы и паттерны, которые позволяют сделать код более читаемым, поддерживаемым, а также снизить порог входа для новых членов команды разработки. Рассмотрим некоторые из них.

Монолитная архитектура [9] – это подход к разработке программного обеспечения, в котором все модули сильно связаны между собой и представляют неделимое целое. В контексте клиент-серверных приложений можно сказать, что монолитная архитектура представляет собой клиентский и серверный код, размещённый в одном проекте, при этом клиентский код неотделим от серверного.

К достоинствам монолитной архитектуры относится скорость и простота разработки, поскольку вся кодовая база находится в одном месте, производительность, поскольку работа всего приложения выполняется в

рамках одного процесса, а также единая среда данных, что устраняет необходимость в сложных механизмах синхронизации данных.

Недостатки монолитной архитектуры проистекают из её же достоинств, а именно сложность масштабирования, поскольку приложение масштабируется как единое целое, что в свою очередь вызывает дополнительное потребление ресурсов, проблемы с производительностью при росте числа пользователей, поскольку код приложения выполняется в рамках одного процесса, а также малое технологическое разнообразие, поскольку внедрение новой технологии может вызвать необходимость в перепроектировании всего приложения, что замедляет дальнейшее развитие.

В современных реалиях монолитную архитектуру стоит рассматривать в качестве архитектурного подхода для небольших проектов с ограниченным жизненным циклом, в корпоративных приложениях, где количество пользователей не превышает десятков или сотен, а также для проектов со сложной предметной областью.

Микросервисная архитектура [9] – это подход к разработке программного обеспечения, при котором программный код делится на отдельные автономные модули. К контексте клиент-серверных приложений микросервисную архитектуру можно рассматривать на разных уровнях, например, отделение клиентской части от серверной, деление клиентской части на микрофронтенды [10], деление серверной части на микросервисы.

Достоинства микросервисной архитектуры напрямую проистекают из главного принципа – деления программного кода на автономные модули, что обеспечивает гибкость и масштабируемость, технологическое разнообразие и расширяемость.

К недостаткам микросервисной архитектуры относится проблема хранения данных и их синхронизация между модулями, что требует использования дополнительных программных средств, например, очереди сообщений, таких как Apache Kafka [11] или RabbitMQ [12], веб-серверов и балансировщиков нагрузки, таких как nginx [13] или Apache HTTP Server [14], а также единого шлюза API (API Gateway), таких как Yandex API Gateway [15] или KrakenD [16].

В современных реалиях микросервисная архитектура используется для больших проектов, которым требуется гибкость и масштабируемость, постоянное внедрение новых функций, при этом у которых простая предметная область и большое количество пользователей. но позволить такую архитектуру могут в большей части крупные компании со своим штатом разработчиков, такие как банки, например, Альфа-Банк [17] или Т-Банк [18], или маркетплейсы, например, Ozon [19] или Wildberries [20].

1.3. Анализ и выбор программных средств разработки

2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ

2.1. Расчёт экономической выгоды

2.2. Проектирование базы данных

3. РАЗРАБОТКА BACKEND-ЧАСТИ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ

3.1. Разработка доменного слоя

3.2. Разработка инфраструктурного слоя

3.3. Разработка слоя бизнес-логики

3.4. Разработка слоя API

ЗАКЛЮЧЕНИЕ

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Разработка через тестирование (TDD) [Электронный ресурс]. URL: <https://doka.guide/tools/tdd/> (дата обращения: 15.02.2026).
2. Abel Avram, Floyd Marinescu. Domain Driven Design Quickly. C4Media, 2006.
3. Смарт Джон Фергюсон, Молак Ян. BDD в действии. ДМК-Пресс, 2023.
4. Официальная документация Cucumber [Электронный ресурс]. URL: <https://cucumber.io/> (дата обращения: 15.02.2026).
5. Официальная документация SpecFlow [Электронный ресурс]. URL: <https://www.specflow.com/> (дата обращения: 15.02.2026).
6. Официальная документация фреймворка Behave [Электронный ресурс]. URL: <https://behave.readthedocs.io/en/latest/> (дата обращения: 15.02.2026).
7. Inside Spec-Driven Development: What GitHub's Spec Kit Makes Possible For AI-assisted Engineering [Электронный ресурс]. URL: <https://www.epam.com/insights/ai/blogs/inside-spec-driven-development-what-githubspec-kit-makes-possible-for-ai-engineering> (дата обращения: 18.02.2026).
8. AI-агенты: как они устроены и что умеют [Электронный ресурс]. URL: <https://cloud.ru/blog/kak-ustroyeny-i-cto-umeyut-ai-agenty> (дата обращения: 18.02.2026).
9. Monolith vs Microservice Architecture: A Comparison [Электронный ресурс]. URL: <https://camunda.com/blog/2023/08/monolith-vs-microservice-architecture-comparison/> (дата обращения: 26.02.2026).
10. Что такое микрофронтенд [Электронный ресурс]. URL: <https://thecode.media/cto-takoe-mikrofrontend/> (дата обращения: 27.02.2026).
11. Официальная документация Apache Kafka [Электронный ресурс]. URL: <https://kafka.apache.org/> (дата обращения: 27.02.2026).

12. Официальная документация RabbitMQ [Электронный ресурс]. URL: <https://www.rabbitmq.com/> (дата обращения: 27.02.2026).
13. nginx [Электронный ресурс]. URL: <https://nginx.org/> (дата обращения: 27.02.2026).
14. Apache HTTP Server Project [Электронный ресурс]. URL: <https://httpd.apache.org/> (дата обращения: 27.02.2026).
15. Официальная документация Yandex API Gateway [Электронный ресурс]. URL: <https://yandex.cloud/ru/services/api-gateway> (дата обращения: 27.02.2026).
16. Официальная документация KrakenD [Электронный ресурс]. URL: <https://www.krakend.io/> (дата обращения: 27.02.2026).
17. Alfa-Digital [Электронный ресурс]. URL: <https://digital.alfabank.ru/techradar/> (дата обращения: 27.02.2026).
18. Т-Банк [Электронный ресурс]. URL: <https://www.tbank.ru/career/it/about/> (дата обращения: 27.02.2026).
19. Ozon Tech [Электронный ресурс]. URL: <https://ozon.tech/> (дата обращения: 27.02.2026).
20. WBTECH [Электронный ресурс]. URL: <https://wbtech.wildberries.ru/> (дата обращения: 27.02.2026).

ПРИЛОЖЕНИЕ

Репозиторий – <https://github.com/Shtskkh/Events.Backend>